3

Partitioning Basics

ORACLE

Copyright © 2005, Oracle. All rights reserved.

Objectives

After completing this lesson, you should be able to do the following:

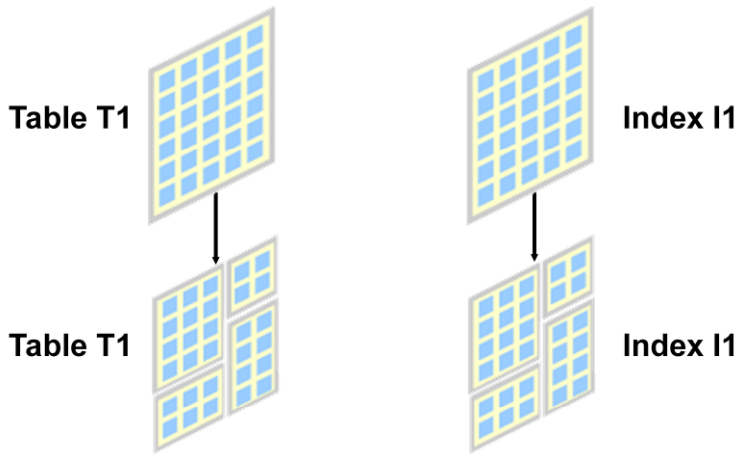
- **Outline the Oracle Database partitioning architecture**
- **Describe the supported partition**
- **List options for creating a partitioned table**
- **Create partitioned tables**

ORACLE

Copyright © 2005, Oracle. All rights reserved.

Partitioned Tables and Indexes

Large tables and indexes can be partitioned into smaller, more manageable pieces.

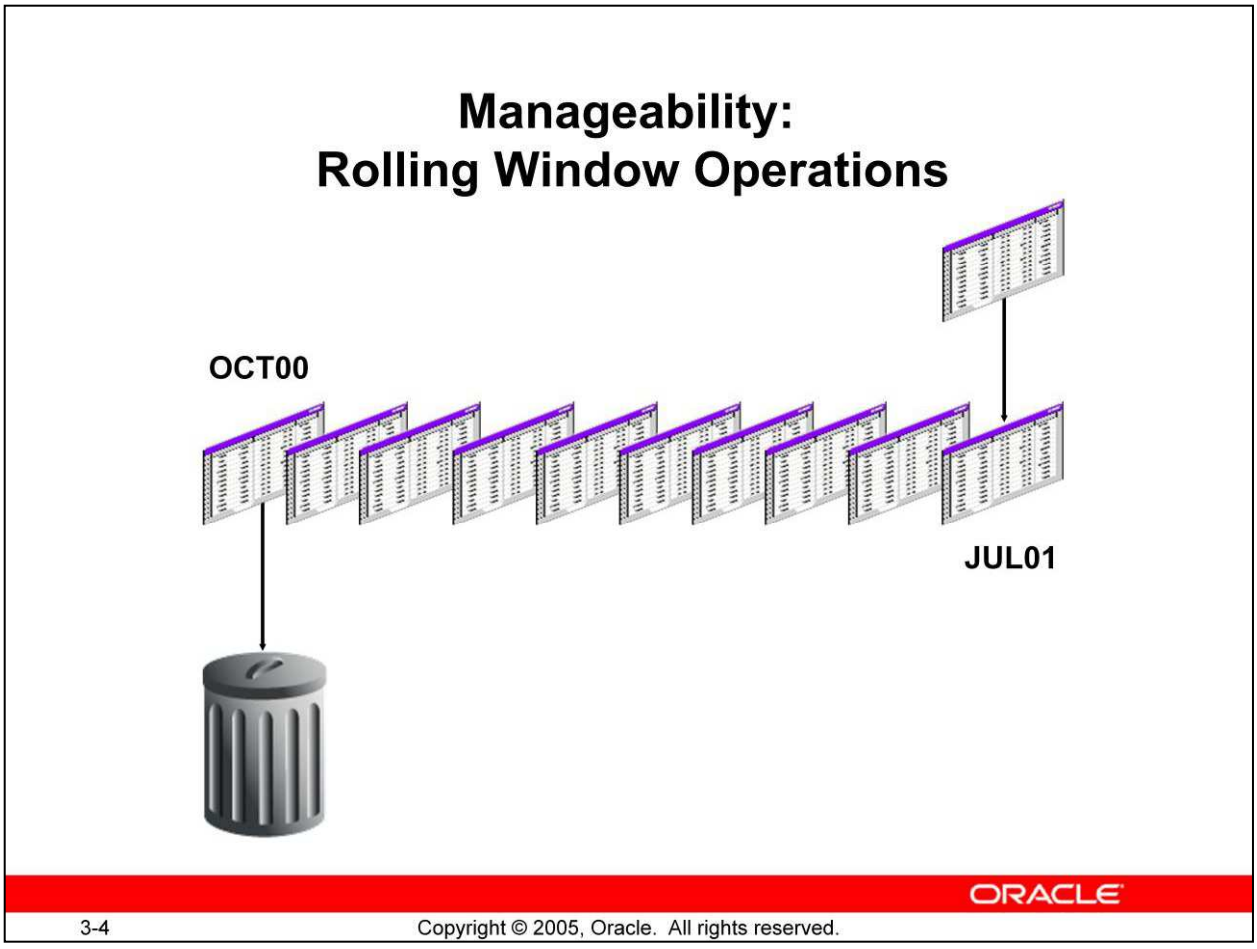


3-3 Copyright © 2005, Oracle. All rights reserved.

Partitioned Tables and Indexes

Partitioned tables allow your data to be broken down into smaller, more manageable pieces called partitions, or even subpartitions. Indexes can be partitioned in a similar fashion. Each partition can be managed individually, and can function independently of the other partitions, thus providing a structure that can be better tuned for availability and performance.

Partitioning is transparent to existing applications as are standard DML statements run against partitioned tables. However, applications can be enhanced to take advantage of partitioning by using partition-extended table or index names in the application DML.



Timeline Databases

Rolling window tables and tables that grow in a linear fashion along business-relevant periods are probably the best examples for using partitions:

- Partitions turn costly deleting of individual rows into simple dictionary operations.
- Adding 10,000 rows to a table can be as simple as adding the extents of an already loaded table to an existing partitioned table, thus converting them to a table partition. Again, this is just a dictionary operation.
- Index management can be automated. Purging October 2000 table partition entries and all relevant index entries can be done with one `ALTER TABLE` statement.
- Creating index entries for the July 2001 table can be done without affecting the existing index entries to the rest of the table.
- DBAs can use `SELECT`, `INSERT`, `UPDATE`, and `DELETE` on individual partitions.

Benefits of Partitioning: Performance Considerations

- **The optimizer eliminates (prunes) partitions that do not need to be scanned.**
- **Partitions can be scanned, updated, inserted, or deleted in parallel.**
- **Join operations can be optimized to join "by the partition."**
- **Partitions can be load-balanced across physical devices.**

ORACLE

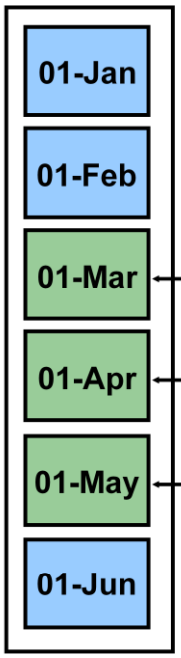
3-5 Copyright © 2005, Oracle. All rights reserved.

Improved Performance

The optimizer is aware of the following points when accessing a partitioned table or index:

- If `WHERE` clauses are specified in a SQL statement, the optimizer can evaluate the statement and based on values, prune partitions that do not need to be accessed.
- Queries and DML operations are narrowed down to partition level instead of full table/index scan.
- Partitionwise joins are used when the tables are partitioned by the join key. This speeds up the join operation, because the amount of data exchanged between query slaves is reduced.
- When the optimizer does sorting, it can apply to partitions instead of to the whole table, causing less temporary sort area in most cases.
- Users can map different partitions to different tablespaces, allowing frequently accessed data to reside on the fastest disks.
- Bulk and maintenance operations can be applied to smaller units of storage.
- Oracle Real Application Clusters can enforce ownership of data by a specific node.

Performance Consideration: Partition Pruning



Sales

Partition pruning: Only the relevant partitions are accessed.

```
SQL> SELECT SUM(amount_sold)
  2  FROM sales
  3  WHERE time_id BETWEEN
  4  TO_DATE('01-MAR-2000',
  5          'DD-MON-YYYY') AND
  6  TO_DATE('31-MAY-2000',
  7          'DD-MON-YYYY');
```


Copyright © 2005, Oracle. All rights reserved.

Partition Pruning

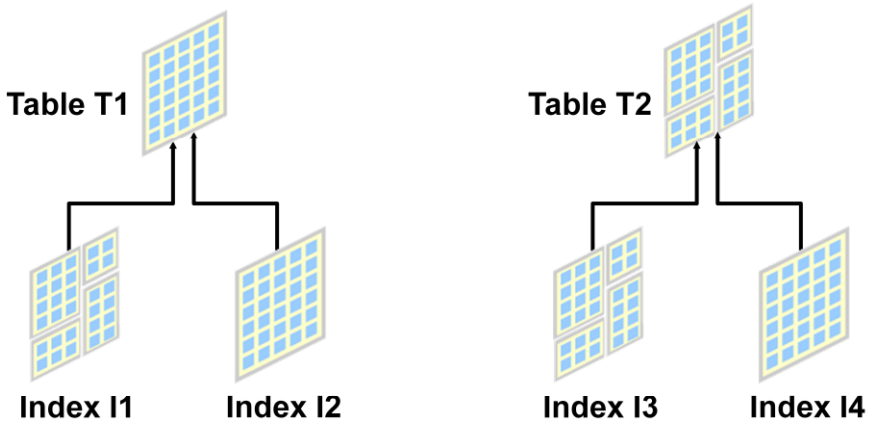
Depending on the SQL statement, the Oracle server can explicitly recognize partitions and subpartitions that need to be accessed and the ones that can be eliminated. This optimization is called partition pruning. This can result in substantial improvements in query performance. However, the optimizer cannot prune partitions if the SQL statement applies a function to the partitioning column.

Pruning is expressed using a range of partitions, and the relevant partitions for the query are all the partitions between the first and the last partition of that range. This allows pruning for conjunctive predicates such as c > 10 and c < 20 but not for disjunctive predicates such as c in (10,30) or (c > 10 and c < :B1) or (c > :B2 and c < 1000).

Table Versus Index Partitioning

A nonpartitioned table can have partitioned or nonpartitioned indexes.

A partitioned table can have partitioned or nonpartitioned indexes.



Copyright © 2005, Oracle. All rights reserved.

Table and Index Partitioning

In general, you can mix partitioned and nonpartitioned indexes with partitioned and nonpartitioned tables.

- A partitioned table can have partitioned and nonpartitioned indexes.
- A nonpartitioned table can have partitioned and nonpartitioned indexes.
- Bitmap indexes on nonpartitioned tables cannot be partitioned.

However, there are design considerations that should be made on the basis of performance, availability, and manageability.

Partitioning Methods

- ***Range*** **partitioning maps data to partitions on the basis of ranges of partition key values for each partition.**
- ***Hash*** **partitioning maps data to partitions by using a hashing algorithm applied to a partitioning key.**
- ***List*** **partitioning maps rows to partitions by using a list of discrete values for the partitioning column.**
- ***Composite*** **partitioning:**
  - ***Range-Hash*** **subpartitions the range partitions using a hashing algorithm.**
  - ***Range-List*** **subpartitions the range partitions using an explicit list.**

ORACLE

Copyright © 2005, Oracle. All rights reserved.

Range Partitioning

Range partitioning uses ranges of column values to map rows to partitions. Range partitions are ordered and this ordering is used to define the lower and upper boundaries of a specific partition. It is the most common type of partitioning and is often used with dates. For example, you may want to partition sales data into monthly partitions. However, it is not always possible to know beforehand how much data will map into a given range, and in some cases, sizes of partitions may differ quite substantially, resulting in suboptimal performance for certain operations such as parallel DML.

Hash Partitioning

This method uses a hash function on the partitioning columns to stripe data into partitions. The hashing algorithm evenly distributes rows among partitions, giving partitions approximately the same size. Hash partitioning is the ideal method for distributing data evenly across devices.

Composite Partitioning

Composite partitioning combines range with hash or list (range-hash, range-list) partitioning. The data is first distributed into partitions according to boundaries established by the partition ranges. For range-hash partitioning, Oracle Database uses a hashing algorithm to further divide the data into subpartitions within each range partition.

Composite Partitioning (continued)

For range-list partitioning, the data is subdivided into subpartitions within each range partition on the basis of the explicit list you choose. This type of partitioning supports historical operations data at the partition level and parallelism (parallel DML) and data placement at the subpartition level.

List Partitioning

The `LIST` method allows explicit control over how rows map to partitions. This is done by specifying a list of discrete values for the partitioning column in the description for each partition.

This partition method allows the modeling of data distributions that follow discrete values that are unordered and unrelated sets of data. These can be grouped and organized together very naturally, using list partitioning.

CREATE TABLE Statement with Partitioning

A partitioned table declaration contains three elements:

- **The logical structure of the table**
- **The partition structure defining the type and columns**
- **The structure of each table partition**

```
SQL> CREATE TABLE example
  2   ( idx NUMBER, txt VARCHAR2(20) )
  3   PARTITION BY RANGE ( idx )
  4   (  PARTITION VALUES LESS THAN ( 0 )
  5        TABLESPACE data01
  6   ,  PARTITION VALUES LESS THAN ( MAXVALUE ));
```

ORACLE

Copyright © 2005, Oracle. All rights reserved.

Creating a Partitioned Table

Consider the example in the slide. It is a range-partitioned table, in which all rows that contain a negative number in the `idx` column are stored in the first partition, which is stored in the `DATA01` tablespace. All other rows, including those with `NULL` in the `idx` column, are stored in the other partition that is stored in the user's default tablespace. Queries on this table have the benefit of partition pruning and other partition-related performance improvements, if appropriate. A partitioned table declaration contains three elements:

- The logical structure of the table
- The partition structure, which defines the type and columns
- The structure of each table partition, which has two parts:
  - The logical bounds
  - The physical storage attributes

Logical and Physical Attributes

Logical attributes:

- **Normal table structure (columns, constraints)**
- **Partition type**
- **Keys and values**
- **Row movement**

Physical attributes:

- **Tablespace**
- **Extent sizes, block attributes**

ORACLE

Copyright © 2005, Oracle. All rights reserved.

Logical and Physical Attributes

When specifying a partitioned table or index, a single statement can specify several attributes. The attributes can easily be divided into those attributes that declare something logical about the table or partition, and those that specify something about the physical storage or manipulation of the table partitions.

Generally, the logical attributes pertain to the table as a whole and are declared first, whereas the physical attributes pertain to each partition. Physical storage attributes declared on the table are used as the default values for the partitions.

There is no change in the way the normal structure of the table is declared when the table is partitioned. The `table_partitioning_clause` can simply be appended to an existing table creation script.

Partitioned tables can also be created with the `AS SELECT` clause.

The storage clauses that can be applied to each table partition are the same storage clauses that can be applied to a normal table, such as `TABLESPACE`, `STORAGE` (`INITIAL`, `NEXT`), and `PCTFREE`. Therefore, storage clause functionality is not explained further in this course.

Partitioning Type

The partition type is declared in the `PARTITION BY` clause:

```
SQL> CREATE TABLE ( … column … )
  2  PARTITION BY RANGE ( column_list )
  3  ( PARTITION specifications ) ;
```

```
  2  PARTITION BY HASH ( column_list )
```

```
  2  PARTITION BY LIST ( column )
```

```
  2  PARTITION BY RANGE ( column_list )
        SUBPARTITION BY HASH ( column_list2 )
```

```
  2  PARTITION BY RANGE ( column_list )
        SUBPARTITION BY LIST ( column )
```

ORACLE

Copyright © 2005, Oracle. All rights reserved.

Partitioning Types

The supported partitioning types are declared in the `PARTITION BY` clause.

Composite partitioning is limited to being a `RANGE` partition on the top level, and a `HASH` or `LIST` partition on the sublevel. The partition key can consist of several columns, analogous to composite column indexes, except for list partitions.

Partitioning can be applied to normal heap organized tables and to index-organized tables (IOTs). IOTs can now be list-partitioned in Oracle Database 10*g*.

Specifying Partition Attributes

Each partition is specified in a partition value clause.

```
   ...
    PARTITION simple_p1 VALUES ( 'HIGH', 'MED'
)
        TABLESPACE data01 PCTFREE 5
  , PARTITION simple_p2 VALUES ( 'LOW' )
        TABLESPACE data02 STORAGE ( INITIAL 1M
)
   ...
```

There can be up to 65,535 partitions per table.

Copyright © 2005, Oracle. All rights reserved.

Specifying Partition Attributes

The slide example shows two partitions being specified in a list-partitioned table. The general structure is a comma-separated list:

```
PARTITION name partition-key-value storage-attributes
```

The partition *name* is optional. If omitted, the system names it `SYS_P`*nnnn*, where *nnnn* is a unique number. Segment names and partition names are distinct. Partition names must be unique only for the table they belong to.

The *partition-key-value* specification must correspond in type and number to the partition key definition. These must be literals, and not be dependent on the environment; for example, format masks.

The *storage-attributes* syntax is the same as used on normal tables, and is optional and separate for each partition. Defaults are taken from the table declaration, if they are listed there; otherwise, they are taken from tablespace or server defaults as usual. There is no requirement that a separate tablespace must be declared for each partition, but it is usually useful.

Partition Key Value

- The partition key value must be a literal.
- Constant expressions are not allowed, with the exception of `TO_DATE` conversion.
- The partition key can consist of up to 16 columns.

ORACLE

3-14 Copyright © 2005, Oracle. All rights reserved.

Partition Key Values

The partition key values must be literals. Even simple expressions are not allowed, such as:

```
3+5, TO_NUMBER('67'), or ASCII('G')
```

The exception is the `TO_DATE` conversion function, which is used to specify a date literal, when the partition key is of the `DATE` type. The purpose is to be able to specify the national language support (NLS) formatting to interpret the date string:

```
TO_DATE('27-12-2002', 'dd-mm-yyyy', 'nls_calendar=gregorian')
```

If this is not done, the `CREATE` statement relies on the NLS environment. Note that the year must be specified with four digits.

Range Partitioning

- Specify the columns to be partitioned, and the break values for each partition.
- Each partition must be defined.
- The `MAXVALUE` value allows the greatest possible value and fits all data types.
- The `MAXVALUE` value includes `NULL` values.

ORACLE

3-15 Copyright © 2005, Oracle. All rights reserved.

Range Partitioning

The partition key can be any columns from the table, within the data type restrictions. Each partition's end points are specified with:

```
VALUES LESS THAN ( value-list )
```

The value list must correspond in type and position to the partition key. These values are noninclusive. That is, the partition key of rows in a partition does not include the value listed.

The `MAXVALUE` value allows the greatest possible value, and fits all data types. Conversely, the smallest possible value will be stored in the first partition. If you omit the partition with the `MAXVALUE` bound, then there is an implied check constraint on the column.

`NULL` values are stored in the partition with the `MAXVALUE` end point. `NULL`s are treated as "one greater than the highest possible value." There is no way to specify `MAXVALUE-1` or otherwise partition `NULL` values separately. You can specify a `NOT NULL` constraint on the partition key column(s).

Range Partitioning: Example

```
CREATE TABLE salestable
(s_productid NUMBER,
s_saledate DATE,
s_custid NUMBER,
s_totalprice NUMBER)
PARTITION BY RANGE(s_saledate)
(PARTITION sal99q4 VALUES LESS THAN (TO_DATE('01-JAN-
2000', 'DD-MON-YYYY')),
PARTITION sal00q1 VALUES LESS THAN (TO_DATE('01-APR-
2000', 'DD-MON-YYYY')),
PARTITION sal00q2 VALUES LESS THAN (TO_DATE('01-JUL-
2000', 'DD-MON-YYYY')),
PARTITION sal00q3 VALUES LESS THAN (TO_DATE('01-OCT-
2000', 'DD-MON-YYYY')),
PARTITION sal00q4 VALUES LESS THAN (TO_DATE('01-JAN-
2001', 'DD-MON-YYYY')));
```


Copyright © 2005, Oracle. All rights reserved.

Range Partitioning: Example

The example in the slide creates the `SALESTABLE` table for a period of five quarters spanning 1999 and 2000, and partitions it by range according to the `S_SALESDATE` column to separate the data into five quarters, each corresponding to a partition.

List Partitioning

```
CREATE TABLE sales_list
(salesman_id NUMBER(5),
salesman_name VARCHAR2(30),
sales_state VARCHAR2(20),
sales_amount NUMBER(10),
sales_date DATE)
PARTITION BY LIST(sales_state)
(PARTITION sales_west VALUES('California',
'Hawaii'),
PARTITION sales_east VALUES ('New York',
'Virginia', 'Florida'),
PARTITION sales_central VALUES('Texas',
'Illinois'),
PARTITION sales_other VALUES(DEFAULT));
```


Copyright © 2005, Oracle. All rights reserved.

List Partitioning

List partitioning enables you to explicitly control how rows map to partitions. You do this by specifying a list of discrete values for the partitioning column in the description for each partition. This is different from range partitioning, where a range of values is associated with a partition, and from hash partitioning, where you have no control of the row-to-partition mapping. The advantage of list partitioning is that you can group and organize unordered and unrelated sets of data in a natural way. The example in the slide creates a list-partitioned table grouping states according to their sales regions.

An additional capability with list partitioning is that you can use a default partition, so that all rows that do not map to any other partition do not generate an error. You can create a default partition as shown in the example.

Hash Partitioning, Named Partitions

Specify the columns to be partitioned, and the number of partitions:

- **Partition may be defined, or just quantified.**
- **NULL is placed in the first partition.**
- **The number of partitions should be a power of two.**

```
CREATE TABLE simple
  (idx NUMBER, txt VARCHAR2(20) PRIMARY KEY)
   ORGANIZATION INDEX
   PARTITION BY HASH ( txt )
 ( PARTITION s_h1 tablespace data01
 , PARTITION s_h2 tablespace data03
  )
;
```

ORACLE

Copyright © 2005, Oracle. All rights reserved.

Hash Partitioning

The partition key can consist of any columns from the table, within the data type restrictions. The partition's end points are not specified. Rows are placed in a partition according to hash values derived from the column values. The hash partitions can be specified with name and tablespace, but with no other attributes. Other storage attributes must thus be defined in the tablespace. Alternatively, the hash partitions are not specified, but only the quantity. NULL values are stored in the first partition. In the example in the slide, rows are evenly distributed in all partitions and the partitions are explicitly named. In the example below, the partitions will be named by the database:

```
CREATE TABLE sales_hash
  (salesman_id NUMBER(5),
   salesman_name VARCHAR2(30),
   sales_amount NUMBER(10),
   week_no NUMBER(2))
PARTITION BY HASH(salesman_id)
PARTITIONS 4;
```

It is recommended that the number of partitions is a power of two, that is, 2, 4, 8, 16, and so on. This is recommended, regardless of which two syntax variations are used to define the hash partitions. If the number is not a power of two, the first few partitions will contain disproportionately more rows due to the hash and partitioning algorithm used.

Hash-Range Partitioning

Hash-Range partitioning example:

```
SQL> CREATE TABLE simple
  2    ( idx NUMBER, txt VARCHAR2(20) )
  3  PARTITION BY RANGE ( idx )
  4    SUBPARTITION BY HASH ( txt )
  5     SUBPARTITIONS 4 STORE IN (data01, data02)
  6  ( PARTITION ns_lo VALUES LESS THAN ( 0 )
  7  , PARTITION ns_hi VALUES LESS THAN ( 1E99 )
  8  , PARTITION ns_mx
  9              VALUES LESS THAN ( MAXVALUE )
 10      SUBPARTITIONS 2 STORE IN ( data03 ) ) ;
```


Copyright © 2005, Oracle. All rights reserved.

Composite Partitioning

Composite partitioning is a hash partitioning of a range-partitioned table partition.

The range partition is specified as a normal range partition type. The hash partition under the range partition is specified with the `SUBPARTITION` clause, but otherwise uses the same syntax as for simple hash partitioning.

The subpartition partition key can be the same or different from the range partition key.

There is no storage clause associated with each range partition, because they are stored as hash subpartitions. You can specify the hash subpartitions for each range partition, thus indirectly giving each range partition different physical attributes.

This example uses the numbered hash partitions variation to specify the subpartitions.

Only rows with `idx` having `NULL` will be in the `ns_mx` subpartitions. (An additional check constraint prohibiting `idx` greater than `1E99` can be added.)

All range partitions are stored in four hash subpartitions in the `data01` and `data02` tablespaces, except the `ns_mx` partition, which uses only two hash subpartitions stored in the `data03` tablespace.